Active Directory HR extractor configuration

Brainwave GRC

December 15, 2025

Contents

Introduction . . . 3
AD Objects extraction . . . 3
- Prerequisites . . . 3
- ICF compatibility . . . 3
- Dependencies . . . 3
- Expected results . . . 3
- Attributes mapping . . . 4
- Extraction command parameters . . . 5

Examples . . . 7
- Example 1 . . . 7
- Example 2 . . . 8
- Example 3 . . . 8

Introduction

This documentation describes how to use scripts provided by Brainwave Identity GRC to extract the human resources data (only users objects) from Active Directory environments, from single domain to the largest multiple forsest/domains configurations.

AD Objects extraction

The provided script ExtractAD.ps1 allows to extract the HR data of users objects managed in Active Directory.

Prerequisites

The extraction script is stored in extractors/bw_activedirectory in the project when the installation of the add-on is done.

- ExtractAD.ps1: Extraction des objets Active Directory

Versions: bw_activedirectory_5.1 add-on

ICF compatibility

This script is compatible with brainwave ICF Powershell connector.

Dependencies

- Requires PowerShell 4.0 or greater
- Domain user with at least “read” privilege
- Requires .NET Framework 4.5 or greater

> **Note:** The powershell script is signed, any modification of the script will make it ineffective unless you change your local security parameters in order to authorize the execution of unsigned scripts. Please note that this modification leads to potential vulnerabilities.

Expected results

ActiveDirectory script generates a CSV file per AD domain. The resulting file is as follows:

- `{DomainNetBiosName}.csv`

The CSV file header is:

```
hrcode, givenname, surname, altname, fullname, titlecode, mail, phone,
    mobile, internal, employeetype, arrivaldate, departuredate, active,
    jobtitlecode, jobtitledisplayname, organisationcode,
    organisationshortname, organisationdisplayname, organisationtype,
    parentorganisationcode, linemanager, managedorgcode, analyticsgroup
```

The script also produces a log file. This file is named `ExtractAD{timestamp}.log` and contains details of extractions according to the `-loglevel` parameter.

Attributes mapping

By default, the script will base on the following attributes mapping between HR attributes (csv file header) and Active Directory attributes.

HR attribute	AD attribute
hrcode	samaccountname
givenname	givenname
surname	sn
altname	
fullname	displayname
titlecode	
mail	mail
phone	
mobile	
internal	
employeetype	employeetype
arrivaldate	createtimestamp
departuredate	accountexpires
Jobtitlecode	title
jobtitledisplayname	title
organisationcode	department
organisationshortname	department
organisationdisplayname	department
organisationtype	
Parentorganisationcode	
linemanager	
managedorgcode	

HR attribute	AD attribute
analyticsgroup	

> **Note:** to customise the default attributes mapping please use the `-customHrAttributes` parameter (please check the examples)

The “active” value is calculated automatically based on “useraccountcontrol” Active Directory attribute and cannot be customised

Extraction command parameters

- **Attribute name:** `-hrData`
- **Mandatory:** Yes
- **Description:** Must be set to `$true` to export users only in csv file to be collected as HR data.

- **Attribute name:** `-customHrAttributes`
- **Mandatory:** No
- **Description:** Path to file that contains custom mapping between HR attributes and AD attributes. The File must have the following format:

hr_attribute	ad_attribute
hrcode	sn
altname	SAMaccountname

The csv delimiter should be `;` and the header must be:

- `hr_attribute`: name of HR attribute, must be part of CSV header
- `ad_attribute`: name of AD attribute, must be a valid AD attribute

The mapping from the csv file will override the default mapping if a difference is found.

- **Attribute name:** `-filter`
- **Mandatory:** No
- **Description:** An LDAP filter by default it is: `(objectcategory=Person)`, script will add automaticaly a filter to extract only users objects, if you provide for example `(cn=*Marketing*)` as value, the final filter will be `(&(objectcategory=Person)((cn=\*Marketing\*)))`

- **Attribute name:** `-server`
- **Mandatory:** No
- **Description:** File that contains in each line a name of an AD server, if the file is not specified, the script will extract all the ActiveDirectory domains data in the current forest. If not set script will try to detect the domain name of current session.

- **Attribute name:** `-credential`
- **Mandatory:** No
- **Description:** File containing PSCredential used to extract objects in ActiveDirectory, see example to create and store PSCredential object to a file, if not set current session credential will be used. If not set the script will use Credential of current session.

- **Attribute name:** `-LogLevel`
- **Mandatory:** No
- **Description:** This parameters allow to specify a log level. The default value is `Error`. Possible values are:
  - **Error**
  - **Warning**
  - **Info**
  - **Debug**

- **Attribute name:** `-errAction`
- **Mandatory:** No
- **Description:** Specify to the script what action to perform if an error occur. The default value is 'Stop'. The possible values are:
  - **Continue** : The script continues if an error occurs and displays it.
  - **SilentlyContinue** : The script runs without displaying errors.
  - **Stop** : The script stops if an error occurs, all errors become critical.

- **Attribute name:** `-printLog`
- **Mandatory:** No
- **Description:** Specify to the script whether to display traces in the console. Possible values are **`true`** and **`false`**. Default value is **`true`**.

- **Attribute name:** `-outputDirectory`
- **Mandatory:** No
- **Description:** Specify to the script where the generated data files will be stored. The default value is the current directory.

- **Attribute name:** `-logDirectory`
- **Mandatory:** No
- **Description:** Specify to the script where the generated log file will be stored. The default value is the current directory.

Examples

Example 1

This example details how to create a PSCredential and store it in a file. The password will automatically be encrypted by powershell API. The generated file can only work using the local machine and user session used to create it.

```
$pwd=ConvertTo-SecureString --String $Password --AsPlainText -Force
$credential=New-Object --TypeName System.Management.Automation.PSCredential
    --ArgumentList $filerLogin, $pwd
$credential | Export-CliXml credential.xml
```

```
$credential = Get-Credential -UserName test\local
$credential | Export-CliXml credential.xml
```

Example 2

An example of the command to use to extract AD users data using the default attribute mappings and filters. This will only extract the users extract only users.

```
./ExtractAD.ps1 -hrData $true -servers 'servers.cfg' -credential credential.
    xml
```

Example 3

An example of the command to use to extract AD users data using custom attribute mappings and filters. To extract only users in specific group.

```
./ExtractAD.ps1 -hrData $true -servers 'servers.cfg' -credential credential.
    xml -filter "(memberOf=CN=Professionnal Services ,OU=Groups,DC=test,DC=
    local)" -customHrAttributes attributesMapping.csv
```